

Efficient Reasoning for LLMs through Speculative Chain-of-Thought

Jikai Wang

Soochow University
risus254@gmail.com

Juntao Li*

Soochow University,
ljt@suda.edu.cn

Lijun Wu

Shanghai AI Laboratory
wulijun@pjlab.org.cn

Min Zhang

Soochow University
minzhang@suda.edu.cn

Abstract

Large reasoning language models such as OpenAI-o1 and Deepseek-R1 have recently attracted widespread attention due to their impressive task-solving abilities. However, the enormous model size and the generation of lengthy thought chains introduce significant reasoning costs and response latency. Existing methods for efficient reasoning mainly focus on reducing the number of model parameters or shortening the chain-of-thought length. **In this paper, we introduce Speculative Chain-of-Thought (SCoT), which reduces reasoning latency from another perspective by accelerated average reasoning speed through large and small model collaboration.** SCoT conducts thought-level drafting using a lightweight draft model. Then it selects the best CoT draft and corrects the error cases with the target model. The proposed thinking behavior alignment improves the efficiency of drafting and the draft selection strategy maintains the prediction accuracy for complex problems. Experimental results on GSM8K, MATH, GaoKao, CollegeMath and Olympiad datasets show that **SCoT reduces reasoning latency by 48%~66% for Deepseek-R1-Distill-Qwen-32B while achieving near-target-model-level performance.** Our code is available at https://github.com/Jikai0Wang/Speculative_CoT.

1 Introduction

Large-scale reasoning language models (e.g., OpenAI-o1 (Jaech et al., 2024) and Deepseek-R1 (DeepSeek-AI et al., 2025)) have exhibited exceptional performance across a broad range of reasoning scenarios. Their superior performance stems from two key factors: massive model scale (e.g., DeepSeek-R1 with 671B parameters) and long chain-of-thought (CoT) (Wei et al., 2022) generation (reaching over 10,000 tokens for complex tasks). However, both factors incur substantial computational overhead, resulting in huge inference costs. In many reasoning systems where the model’s reasoning process remains implicit, excessive reasoning time induces significant response latency.

Recent works (Paliotta et al., 2025; Liao et al., 2025; Xu et al., 2025b; Zhang et al., 2025) mainly tackle this challenge by developing smaller reasoning models or reducing the generation length of CoTs. Distillation (Paliotta et al., 2025; Liao et al., 2025) serves as a commonly used technique for transferring strong reasoning capabilities from large reasoning models to small ones. Representatively, Deepseek-R1-Distill-Qwen-1.5B (DeepSeek-AI et al., 2025) exhibits robust performance on diverse datasets. Approaches such as StepSkip (Liu et al., 2024) and TokenSkip (Xia et al., 2025) suggest training reasoning models on compressed CoT data to shorten the CoT length during inference. In

*Corresponding author.

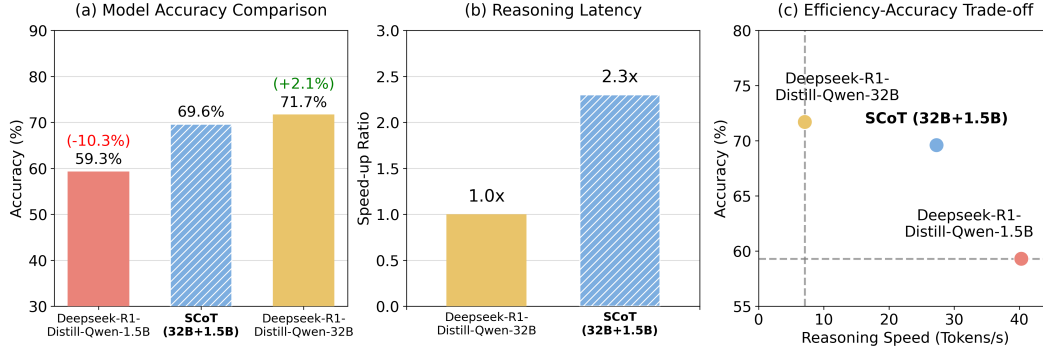


Figure 1: Average reasoning accuracy and efficiency of Speculative Chain-of-Thought (SCoT) with Deepseek-R1-Distill-Qwen-32B and Deepseek-R1-Distill-Qwen-1.5B on GSM8K, MATH, GaoKao, CollegeMath and Olympiad. Graph (a): SCoT achieves near-target-model performance level on the 5 datasets. Graph (b): SCoT demonstrates an average speed-up ratio of 2.3x for reasoning latency. Graph (c): Comparison of reasoning efficiency-accuracy trade-off.

practical applications, reasoning models need to solve problems with widely varying complexity levels. Fundamentally, employing large reasoning models to handle simple tasks is a waste. On the other hand, using small models alone or compressing the chain-of-thought length may compromise performance on complex tasks. Now that we have a powerful and efficient small model, why not prioritize this lightweight alternative and only invoke the large model for deeper thinking when the problem exceeds the capability threshold of the small one?

Based on this motivation, we propose Speculative Chain-of-Thought (SCoT), a reasoning framework that reduces reasoning latency through the collaboration of large and small models. The core idea of SCoT is to use a lightweight model to efficiently generate multiple CoT drafts which are then picked and used by the target model. SCoT is inspired by speculative decoding (Stern et al., 2018; Xia et al., 2023; Leviathan et al., 2023), which performs token-level speculation, whereas our approach conducts thought-level speculation. Unlike shortening the length of CoT, SCoT accelerates the thinking process from another dimension, that is, by increasing the speed of CoT generation.

In Section 3, we introduce thinking behavior alignment to improve drafting efficiency. We also design an error correction mechanism to preserve accuracy in dealing with complex tasks. We construct a training set by 500 samples from GSM8K for fine-tuning the target model to improve the capability of draft selection and error detection. We conduct comprehensive experiments in Section 4 to evaluate the effectiveness of SCoT. Figure 1 displays the overview reasoning accuracy and efficiency of SCoT. Adapting to task difficulty and draft quality, the draft selection mechanism decides when the target model should rethink. Simple tasks mainly utilize the draft model for thought chain generation, while complex problems engage the target model more extensively in the reasoning process. SCoT reduces reasoning latency by 48%~66% on 5 datasets (including the Olympiad-level dataset (He et al., 2024)) while maintaining near-target-model-level performance.

2 Related Works

Efficient Reasoning Reasoning LLMs (Jaech et al., 2024; DeepSeek-AI et al., 2025) generate lengthy CoTs to enhance performance, albeit at the cost of increased latency. Some approaches (Ding et al., 2024; Luo et al., 2025; Xia et al., 2025; Xu et al., 2025b; Zhang et al., 2025) speed up the thinking process by reducing the length of the CoT at the token level. Liu et al. (2024) and Xia et al. (2025) use compressed CoTs to fine-tune the reasoning LMs. Hou et al. (2025) prunes long CoTs through reinforcement learning. Xu et al. (2025a) and Aytes et al. (2025) guide the model to think efficiently through prompting. Soft CoT methods (Saunshi et al., 2025; Xu et al., 2025b; Hao et al., 2024) use latent space representations to replace reasoning tokens, thus reducing the excessive redundancy in the reasoning process. Another mainstream approach (Yu et al., 2024; Paliotta et al., 2025; Liao et al., 2025) for efficient reasoning is to teach smaller models to think. Paliotta et al. (2025) and Liao et al. (2025) transfer reasoning ability from large models to small ones by distillation. Liu et al. (2025) study the impact of quantization for reasoning LLMs. Our approach leverages

collaborative inference between large and small models, combining the small model’s efficiency with the large model’s accuracy to achieve an improved efficiency-accuracy trade-off in reasoning LLMs.

Speculative Decoding Autoregressive language models generate one token in one decoding step, which greatly limits its decoding efficiency. Speculative decoding (Stern et al., 2018; Xia et al., 2023; Leviathan et al., 2023; Chen et al., 2023a) adopts lightweight models for drafting and target models for parallel verification to generate multiple tokens in one decoding step, thus achieving lossless acceleration. The draft model can be an independent small model (Leviathan et al., 2023; Chen et al., 2023a; Spector and Re, 2023; Chen et al., 2023b), a submodule of the target model (Cai et al., 2024; Zhang et al., 2023; Li et al., 2024), or an external corpus (He et al., 2023). Draft tokens that match the original output of the target model will be accepted under a strict verification strategy. Most speculative decoding methods perform token-level drafting, while we borrow the idea of speculation to perform thought-level drafting to improve the reasoning efficiency of LLMs.

3 Speculative CoT

This section introduces Speculative Chain-of-Thought (SCoT), an algorithm for accelerating LLM reasoning. SCoT employs a small model to draft reasoning chains while utilizing the target model to select the best draft for generating the final answer. The overview of SCoT is shown in Figure 3.

3.1 Formulation

For question q in the given question set Q , the reasoning model M first enters the thinking stage after inputting q . Specifically, M will output a chain-of-thought $T = (y_1, y_2, \dots, y_l)$ starting and ending with pre-trained special tokens, where y_i represents the i -th output token and l is length of T . Take Deepseek-R1 (DeepSeek-AI et al., 2025) as an example. The thinking process starts with “<think>” and ends with “</think>”. Next, the model generates the final answer A based on q and T . Our goal is to reduce the inference overhead of T while ensuring the quality of the final answer.

3.2 Generating Drafts of Thoughts

First, we start with a question: “*Can small models generate high-quality chain-of-thoughts?*”. We sampled 500 samples in GSM8K (Cobbe et al., 2021) to evaluate the reasoning accuracy of Deepseek-R1-Distill-Qwen-1.5B and Deepseek-R1-Distill-Qwen-32B (DeepSeek-AI et al., 2025). Figure 2 displays the results. Under the greedy sampling method, the performance of the 1.5B model is significantly worse than that of the 32B model. However, if we allow the 1.5B model to provide 5 answers under the nuclear sampling method and select the best one among the 5 replies for each sample, its overall accuracy on GSM8K is close to that of the 32B model.

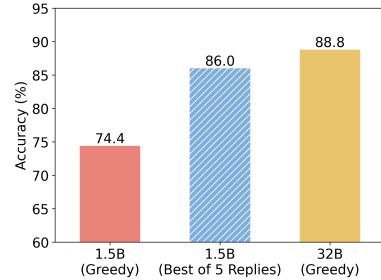


Figure 2: Comparison of reasoning accuracy on GSM8K.

From the above experiments, we can conclude that the small model has the potential to generate high-quality thought chains, but a stronger verifier is needed to control or select its generated results. Therefore, in order to accelerate the thinking process of reasoning LLMs, we first use a smaller model M_d to draft n chains of thoughts in parallel:

$$[T_d^1, T_d^2, \dots, T_d^n] = M_d(\underbrace{[q, q, \dots, q]}_n). \quad (1)$$

We use nuclear sampling for the small model thinking to enhance the diversity of drafts of CoTs. Thanks to their smaller parameter size, smaller models can draft reasoning chains significantly faster than large models when the length of the CoT is the same, thus speeding up the thinking process.

3.3 Thinking Behavior Alignment

Although small models have higher throughput than large ones, significant differences in their reasoning behavior may still result in limited, or even reduced, reasoning speed. In our experiments,

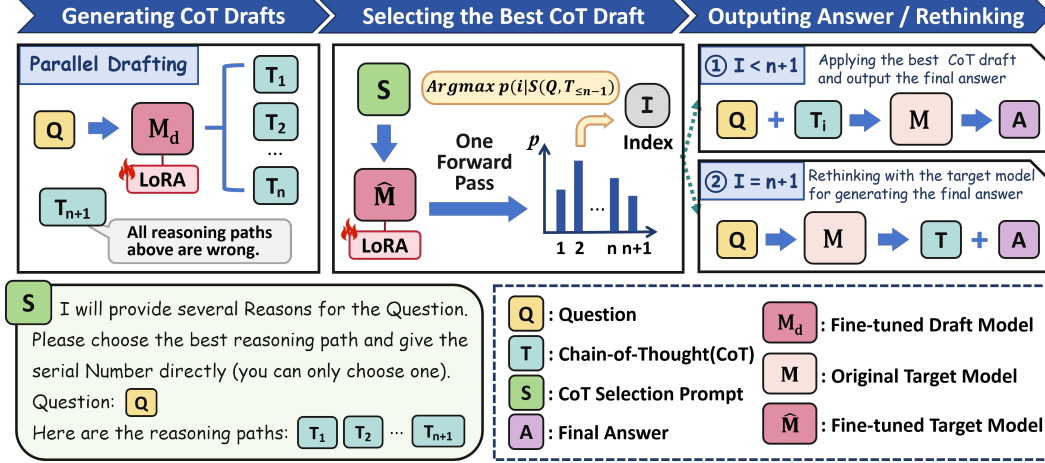


Figure 3: Overview of Speculative Chain-of-Thought. Given a question Q , SCoT first applies a lightweight draft model to generate multiple CoT drafts in parallel. The draft model is fine-tuned with LoRA modules to align the thinking behavior of the target model. It appends a special CoT option for the case where all drafts are wrong. The target model for selecting the best CoT draft is also fine-tuned with LoRA modules for improved accuracy. With the designed prompt template S , only one forward propagation is needed to get the index of the best CoT. Once the best CoT is picked, SCoT directly adopts it for the original target model to generate the final answer. Only if no draft is selected, SCoT will rethink with the target model to ensure the quality of the generated answer.

we observed that even models of the same series can exhibit significant differences in thinking behavior. The inference speed of Deepseek-R1-Distill-Qwen-1.5B is about 6 times that of Deepseek-R1-Distill-Qwen-32B. However, the average CoT length of the 1.5B model on the GSM8K dataset is more than 4 times that of the 32B model. The CoT of the 1.5B model contains more redundant tokens and repeated reflections. Therefore, the draft model requires training in alignment of thinking behavior to ensure drafting efficiency.

We adopt 1500 samples from the GSM8K training set to make the target model generate the corresponding thought chains. The training data $d = (x_1, x_2, \dots, x_m, y_1, y_2, \dots, y_l)$ is a combination of the question and generated CoT, where m is the length of the question and $x_{\leq m}$ are input tokens. Then we use LoRA modules (Hu et al., 2022) to fine-tune the draft model with the cross-entropy loss:

$$\mathcal{L}_{Draft} = -\frac{1}{l} \sum_{i=1}^l \log p_{M_d}(y_i | x_{\leq m}, y_{< i}), \quad (2)$$

where $p_{M_d}(\cdot | \cdot)$ represents the output probability distribution of M_d . The original parameters of the draft model are frozen during training. The LoRA modules are adapted to the Q and V matrices in the attention block of each layer. After training, we merge the LoRA module parameters into the weight matrices of the draft model. Therefore, there is no additional overhead during inference.

3.4 Draft Selection and Error Correction

After generating CoT drafts $(T_1, T_2, \dots, T_n)$ by the fine-tuned draft model, we design a prompt template S for selecting these thought chains. Figure 3 displays the specific content of this template. For particularly challenging problems where all CoT drafts may be incorrect, we aim for the target model to detect such cases to ensure the accuracy of the final answer. For this purpose, we introduce a special option T_{n+1} , which indicates that all CoT drafts are wrong. Through a single forward propagation, we can efficiently obtain the CoT index with the highest probability from the output distribution of the next token by the target model:

$$index = \arg \max_{i \in \mathcal{V}} P_M(i | S(q, T_{\leq n+1})), \quad \mathcal{V} = \{1, 2, \dots, n+1\}. \quad (3)$$

To improve the accuracy of the target model in selecting the correct CoT drafts and detecting errors, we also use the LoRA modules to fine-tune the target model. We use 500 samples from the GSM8K

training set to construct the training data. For each sample, we use the draft model to generate n thought chains and deploy template S as the input. We judge the correctness of the CoT drafts based on the ground truth and construct the label set $\mathcal{Y}$, which is the set of indices for all correct drafts. See Appendix A for examples. We design the following loss function to fine-tune the target model:

$$\mathcal{L}_{Target} = \min\{-\log p_M(y|S(q, T_{\leq n+1}))|y \in \mathcal{Y}\}. \quad (4)$$

Likewise, the original model parameters are frozen, and parameters in LoRA modules are merged into the weight matrices of $\mathbf{Q}$ and $\mathbf{V}$. The fine-tuned target model is only used for CoT selection.

There are two situations when selecting CoT. The first is that the output $index < n + 1$, that is, the target model selects the best one T_{index} among the CoTs generated by the draft model. In this case, we directly let the target model generate the final answer based on q and T_{index} . The second situation is the output $index = n + 1$, which indicates that the problem is too hard and the draft model failed to produce correct reasoning. To maintain performance in the face of these complex problems, we instruct the target model to rethink the question and generate the final answer.

4 Experiments

4.1 Settings

We adopt Deepseek-R1-Distill-Qwen-32B (DeepSeek-AI et al., 2025) (abbreviated as D-Qwen-32B) as the target model and Deepseek-R1-Distill-Qwen-1.5B as the draft model. We use a single A100-PCIE-40GB GPU for the 1.5B model and 4 A100-PCIE-40GB GPUs for the 32B model. We evaluate the efficiency and accuracy of Speculative CoT on reasoning datasets of various difficulty levels, including GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), GaoKao-En-2023 (Liao et al., 2024), CollegeMath (Tang et al., 2024) and Olympiad (He et al., 2024). As stated in Section 3, we use data from GSM8K training set to fine-tune the models with LoRA modules. Note that there is no overlap in the data used for training and evaluation. We set the rank of LoRA modules to 8 for both the draft and target model. The number of CoT drafts n is set to 5. To avoid running out of memory when selecting the drafts, the maximum draft length is set to 5000. We compare the reasoning latency and accuracy of Speculative CoT with the vanilla reasoning by the 32B model.

4.2 Main Results

Table 1: Comparison of CoT length and reasoning efficiency. We report average CoT length of the target model l_M , average CoT length of the draft model l_{M_d} and average CoT latency of each sample $t(s)$. Speed-up ratio r is calculated by average thinking time compared with vanilla decoding.

Method	GSM8K		MATH		GaoKao		CollegeMath		Olympiad	
	l_M	l_{M_d}	l_M	l_{M_d}	l_M	l_{M_d}	l_M	l_{M_d}	l_M	l_{M_d}
D-Qwen-32B	302	-	2123	-	3230	-	1051	-	8431	-
SCoT(32B+1.5B)	45	265	151	1855	630	2767	110	1136	3553	4194
	t	r	t	r	t	r	t	r	t	r
D-Qwen-32B	26.5	1.00	225.1	1.00	369.5	1.00	95.3	1.00	1092.2	1.00
SCoT(32B+1.5B)	11.8	2.26	77.0	2.92	160.1	2.31	43.7	2.18	575.7	1.90

Lower Reasoning Latency Table 1 demonstrates the average CoT length and the average CoT latency of each sample on the 5 datasets. l_M and l_{M_d} represent the numbers of tokens generated by the target model and the draft model during the thinking stage, respectively. Since the draft model generates 5 draft chains in parallel, l_{M_d} refers to the length of the longest draft CoT. The average latency t is measured in seconds. For the 32B model, l_M exhibits a wide range of 302 to 8431 across different datasets. Except for MATH where the average CoT length of SCoT ($l_M + l_{M_d}$) is shorter than that of the 32B model, the overall CoT lengths of the two methods are close. For SCoT, most of the reasoning tokens are generated by the draft model, and the CoT tokens generated by the target model account for 8.8%~45.9% of the total CoT tokens. As task difficulty increases, small

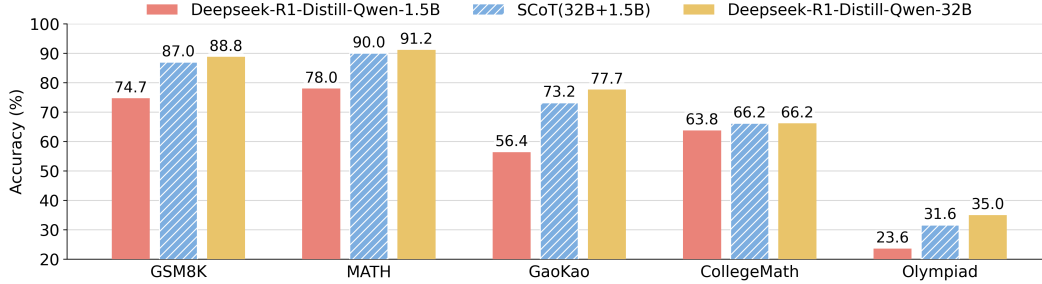


Figure 4: Comparison of accuracy of the final answer on the 5 datasets.

models exhibit higher error rates, thus necessitating greater involvement from the target model in the reasoning process. The draft selection mechanism dynamically determines whether the target model should reconsider its output, evaluating both problem complexity and draft generation quality. For simpler tasks, thought chains are predominantly produced by the draft model. Conversely, the system allocates more thought chain generation to the target model when handling complex problems.

The throughput of the 1.5B model is about 5 times that of the 32B model in our experimental environment. The CoT latency of SCoT is mainly composed of the target model reasoning time, draft model reasoning time, and draft selection time. For the GSM8K dataset, the proportions are approximately 33.9%, 60.2%, and 5.9% respectively. Generally, SCoT achieves greater speedup ratios on simpler tasks where the CoT drafts demonstrate higher quality. SCoT achieves a speedup ratio ranging from 1.90 $\times$ to 2.92 $\times$ compared to vanilla reasoning.

Near-target-model Level Accuracy Figure 4 displays the accuracy of SCoT on these datasets. Comparative results indicate that SCoT maintains near-target-model performance levels on all datasets (even for the Olympiad-level task), with clearly superior accuracy compared to standalone draft model reasoning. For CollegeMath, SCoT even achieves a lossless acceleration.

4.3 Ablation Studies

Table 2: Results for ablation study. Best results are in bold.

Method	GSM8K	MATH	GaoKao	CollegeMath	Olympiad
SCoT (32B+15B)	87.0	90.0	73.2	66.2	31.6
w Single Draft	85.6	86.6	69.9	63.8	30.0
w/o Tuning M	85.6	90.0	69.9	64.8	30.0
w/o Error Correction	85.6	90.0	71.4	65.6	30.2

Table 3: Comparative analysis of average CoT draft length on DeepSeek-R1-Distill-Qwen-1.5B with versus without thinking behavior alignment training.

Method	GSM8K	MATH	GaoKao	CollegeMath	Olympiad
Deepseek-R1-Distill-Qwen-1.5B	606	2092	2589	1355	4032
+ Thinking Behavior Alignment	193	1183	1487	709	3554

We conduct an ablation study in this section. We carry out 3 sets of experiments to observe the changes in accuracy on the 5 datasets with Deepseek-R1-Distill-Qwen: 1) SCoT with Single Draft: Only one CoT draft is generated for each sample (except for the special draft T_{n+1}), 2) SCoT without Tuning M : We use the original target model instead of the fine-tuned version to select the drafts, 3) SCoT without Error Correction: We remove T_{n+1} and the rethinking strategy. Table 2 shows the results. SCoT exhibits the highest accuracy on all datasets, indicating that both the three proposed techniques contribute to the reasoning performance.

In order to verify the effectiveness of thinking behavior alignment, we compare the average CoT draft length generated by the original Deepseek-R1-Distill-Qwen-1.5B and the fine-tuned version. Table 3 shows the mean CoT length on the 5 datasets. The CoT length generated by the fine-tuned model is significantly shorter than that of the original model on all datasets, especially for GSM8K. Therefore, thinking behavior alignment improves the efficiency of the drafting process.

4.4 Draft Selection Accuracy Analysis

In this section, we study the draft selection and error detection accuracy of the fine-tuned Deepseek-R1-Distill-Qwen-32B on the 500 samples in the test set of GSM8K. We divide these samples into two categories according to their labels. Samples for which the draft model can generate the correct reasoning chain form Class 1 (the label $y \neq n + 1$), while others constitute Class 2 (the label $y = n + 1$). Table 4 shows the statistical results. There are 450 first-class samples out of the 500 samples. Since the second type of sample is more difficult to predict and the training data is limited, the target model demonstrates more accurate prediction for Class 1 samples than Class 2 samples. Our preliminary tests show that direct prediction by the original target model on the second-class samples yields zero accuracy. After fine-tuning, the model can correctly predict the labels for about half of the Class 2 samples.

Table 4: The number of two types of CoT drafts and the accuracy of draft selection on GSM8K.

Type	Class 1	Class 2
Amount	450	50
Accuracy	85.1	52.0

4.5 Impact on Length of the Final Answer

We study the impact of SCoT on the length of the final answer in this section. Table 5 displays the answer length of the original 32B model and SCoT on the 5 datasets. In addition to reducing the latency of the reasoning phase, SCoT also reduces the length of the generated final answer compared to the target model and even brings additional inference efficiency improvements.

Table 5: Average length of the final answer generated by Deepseek-R1-Distill-Qwen-32B and SCoT.

Method	GSM8K	MATH	GaoKao	CollegeMath	Olympiad
Deepseek-R1-Distill-Qwen-32B	314	632	948	416	2952
SCoT (32B+1.5B)	270	358	610	357	2179

5 Conclusion

In this paper, we proposed a novel reasoning framework called Speculative Chain-of-Thought to alleviate the latency bottleneck of reasoning LLMs. The proposed method leverages the efficiency of small models for parallel CoT drafting and the higher accuracy of the large model for drafting selection and error correction. The draft selection mechanism adaptively decides whether the target model needs to rethink based on the difficulty of the problem and the quality of the draft generation. Thought chains are mostly generated by the draft model for simple tasks. While more thought chains will be allocated to the target model to generate for complex problems. Speculative Chain-of-Thought demonstrates a speed-up ratio of up to 2.92× for the reasoning process while achieving near-target-model-level accuracy on 5 datasets of different difficulties. It exhibits the potential for future work to realize a better accuracy-efficiency trade-off for reasoning LLMs through model collaboration.

References

- Aytes, S. A., Baek, J., and Hwang, S. J. (2025). Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. (2024). Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. (2023a). Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Chen, Z., Yang, X., Lin, J., Sun, C., Huang, J., and Chang, K. C.-C. (2023b). Cascade speculative drafting for even faster llm inference. *arXiv preprint arXiv:2312.11462*.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Ding, M., Liu, H., Fu, Z., Song, J., Xie, W., and Zhang, Y. (2024). Break the chain: Large language models can be shortcut reasoners. *arXiv preprint arXiv:2406.06580*.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. (2024). Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., Liu, J., Qi, L., Liu, Z., and Sun, M. (2024). OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand. Association for Computational Linguistics.
- He, Z., Zhong, Z., Cai, T., Lee, J. D., and He, D. (2023). Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- Hou, B., Zhang, Y., Ji, J., Liu, Y., Qian, K., Andreas, J., and Chang, S. (2025). Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*.

- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Jaech, A., Kalai, A., Lerer, A., Richardson, A., El-Kishky, A., Low, A., Helyar, A., Madry, A., Beutel, A., Carney, A., et al. (2024). Openai o1 system card. *arXiv preprint arXiv:2412.16720*.
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. (2024). Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*.
- Liao, H., He, S., Hao, Y., Li, X., Zhang, Y., Zhao, J., and Liu, K. (2025). SKIntern: Internalizing symbolic knowledge for distilling better CoT capabilities into small language models. In Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B. D., and Schockaert, S., editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3203–3221, Abu Dhabi, UAE. Association for Computational Linguistics.
- Liao, M., Luo, W., Li, C., Wu, J., and Fan, K. (2024). Mario: Math reasoning with code interpreter output – a reproducible pipeline. *arXiv preprint arXiv:2401.08190*.
- Liu, R., Sun, Y., Zhang, M., Bai, H., Yu, X., Yu, T., Yuan, C., and Hou, L. (2025). Quantization hurts reasoning? an empirical study on quantized reasoning models. *arXiv preprint arXiv:2504.04823*.
- Liu, T., Guo, Q., Hu, X., Jiayang, C., Zhang, Y., Qiu, X., and Zhang, Z. (2024). Can language models learn to skip steps? In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Luo, H., Shen, L., He, H., Wang, Y., Liu, S., Li, W., Tan, N., Cao, X., and Tao, D. (2025). O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*.
- Paliotta, D., Wang, J., Pagliardini, M., Li, K. Y., Bick, A., Kolter, J. Z., Gu, A., Fleuret, F., and Dao, T. (2025). Thinking slow, fast: Scaling inference compute with distilled reasoners. *arXiv preprint arXiv:2502.20339*.
- Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. (2025). Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*.
- Spector, B. and Re, C. (2023). Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*.
- Stern, M., Shazeer, N., and Uszkoreit, J. (2018). Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31.
- Tang, Z., Zhang, X., Wang, B., and Wei, F. (2024). Mathscale: Scaling instruction tuning for mathematical reasoning. *arXiv preprint arXiv:2403.02884*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.
- Xia, H., Ge, T., Wang, P., Chen, S.-Q., Wei, F., and Sui, Z. (2023). Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3909–3925.
- Xia, H., Li, Y., Leong, C. T., Wang, W., and Li, W. (2025). Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*.
- Xu, S., Xie, W., Zhao, L., and He, P. (2025a). Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*.
- Xu, Y., Guo, X., Zeng, Z., and Miao, C. (2025b). Softcot: Soft chain-of-thought for efficient reasoning with llms. *arXiv preprint arXiv:2502.12134*.

- Yu, P., Xu, J., Weston, J., and Kulikov, I. (2024). Distilling system 2 into system 1. *arXiv preprint arXiv:2407.06023*.
- Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., and Mehrotra, S. (2023). Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*.
- Zhang, J., Zhu, Y., Sun, M., Luo, Y., Qiao, S., Du, L., Zheng, D., Chen, H., and Zhang, N. (2025). Lightthinker: Thinking step-by-step compression. *arXiv preprint arXiv:2502.15589*.

A Sample Presentation

We display two samples of constructed training data for draft selection in Figure 5 and Figure 6.

Input: "I will provide several Reasons for the Question. Please choose the best reasoning path and give the serial Number directly (you can only choose one). \n\nQuestion: The rug is 5 feet wider than the chair. The couch is 2 feet longer than twice the width of the rug. If the chair is 3 feet wide. How many feet long is the couch?\n\nHere are the reasoning paths:\n\nReason 1: First, I determine the width of the rug. Since the rug is 5 feet wider than the chair and the chair is 3 feet wide, the rug's width is 3 feet plus 5 feet, which equals 8 feet.\n\nNext, I calculate the length of the couch. The couch is 2 feet longer than twice the rug's width. Twice the rug's width is 2 times 8 feet, which is 16 feet. Adding 2 feet to this gives a total length of 18 feet for the couch.\n\nReason 2: First, identify the given information. The chair is 3 feet wide, the rug is 5 feet wider than the chair, and the couch is 2 feet longer than twice the width of the rug.\n\nNext, calculate the width of the rug by adding 5 feet to the width of the chair. Then, determine the length of the couch by taking twice the width of the rug and adding 2 feet to it.\n\nFinally, perform the arithmetic operations to find the exact length of the couch.\n\nReason 3: First, I know that the chair is 3 feet wide. Next, since the rug is 5 feet wider than the chair, I calculate the rug's width by adding 5 feet to the chair's width. Then, the couch is 2 feet longer than twice the rug's width. I first find twice the rug's width and then add 2 feet to it to determine the couch's length. Finally, by following these steps, I can find the length of the couch.\n\nReason 4: First, I'll note the given information: the width of the chair is 3 feet. The rug is 5 feet wider than the chair, so the width of the rug is 3 feet plus 5 feet, which equals 8 feet. Next, the couch is 2 feet longer than twice the width of the rug. Since the rug is 8 feet wide, twice that width is 16 feet. Adding 2 feet to this gives a total length of 18 feet for the couch. Therefore, the couch is 18 feet long.\n\nReason 5: BPM: Read the problem and identify the given information and what is being asked. The rug is 5 feet wider than the chair. The chair is 3 feet wide. The couch is 2 feet longer than twice the width of the rug. We need to find the length of the couch. First, let's figure out the width of the rug. The rug is 5 feet wider than the chair. Since the chair is 3 feet wide, the rug's width should be $3 + 5 = 8$ feet. Next, the couch is 2 feet longer than twice the width of the rug. So, twice the rug's width is $2 * 8 = 16$ feet. Adding 2 feet to that gives $16 + 2 = 18$ feet. Therefore, the couch is 18 feet long.\n\nReason 6: All reasoning paths above are wrong.\n\nAssistant: >Number of the best reasoning path: "
Label: [1, 4, 5]

Figure 5: The selection template is in red and question is in blue.

Input: "I will provide several Reasons for the Question. Please choose the best reasoning path and give the serial Number directly (you can only choose one). \n\nQuestion: Steve loves playing video games. His parents get him a console along with 5 games for his birthday. He saves up enough money to buy 1 game per month for a year, and then the following year he starts buying 2 games a month. For the third year he buys 4 games a month as he has a new part-time job that makes him more money. He also gets 5 games for Christmas every year. How many games does Steve have after 3 years?\n\nHere are the reasoning paths:\n\nReason 1: To determine how many games Steve has after 3 years, I'll break down his monthly purchases and annual gifts separately. In the first year, Steve buys 1 game per month, totaling 12 games. In the second year, he increases his purchase rate to 2 games per month, resulting in 24 games. In the third year, his purchase rate rises to 4 games per month, adding 48 games. Additionally, Steve receives 5 games at each Christmas, adding 5 games annually. By the end of 3 years, Steve has accumulated a total of 108 games.\n\nReason 2: First, I'll determine how many games Steve buys in the first year. He saves enough to buy 1 game each month for 12 months, totaling 12 games. Next, in the second year, he increases his monthly purchases to 2 games per month. Over 12 months, that amounts to 24 games. In the third year, his buying rate increases to 4 games per month. Again, over 12 months, that's 48 games. Additionally, Steve receives 5 games as Christmas each year, so that's 5 games per year for 3 years, totaling 15 games. Finally, I'll add up all the games: 12 (first year) + 24 (second year) + 48 (third year) + 15 (Christmas gifts) to find the total number of games Steve has after 3 years.\n\nReason 3: First, identify the initial number of games Steve has. He starts with 5 games from his birthday. Next, in the first year, he buys 1 game each month. There are 12 months in a year, so he buys 12 games. Adding this to his initial 5 games gives him 17 games after the first year. In the second year, he increases his purchase rate to 2 games per month. Over 12 months, this results in 24 additional games, bringing the total to 41 games. In the third year, his purchasing rate doubles again to 4 games per month. This year, he buys 48 games, bringing the total to 89 games after three years. Additionally, Steve receives 5 games as Christmas each year. Over three years, this amounts to 15 more games. Finally, add the Christmas games to the previous total to get the final number of games Steve has after 3 years.\n\nReason 4: First, I need to determine how many games Steve buys each year. In the first year, he buys 1 game per month for 12 months, totaling 12 games. In the second year, he increases his buying rate to 2 games per month, so that's 24 games. In the third year, he increases his rate further to 4 games per month, resulting in 48 games. Additionally, Steve receives 5 new games at each Christmas, so that's 5 games annually. Adding all these together, the total number of games Steve has after 3 years is $12 + 24 + 48 + 5 = 89$ games.\n\nReason 5: First, Steve buys 1 game per month for the first year, which is 12 games. In the second year, he increases his monthly game purchases to 2 games, totaling 24 games. In the third year, his purchasing rate doubles again to 4 games per month, totaling 48 games. Additionally, Steve receives 5 games at the end of each year as Christmas gifts, adding 15 games over the three years. Finally, by the end of the third year, Steve has a total of 84 games.\n\nReason 6: All reasoning paths above are wrong.\n\nAssistant: >Number of the best reasoning path: "
Label: [6]

Figure 6: The selection template is in red and question is in blue.